UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO : 202650 abril 2026 – agosto 2026

ASIGNATURA : Modelos discretos para la ing

TEMA : Importancia de la complejidad algorítmica en desarrollo de software

SUBTEMA : Complejidad algorítmica.

ESTUDIANTE : Jeffry Vera, Darlin Demera

NIVEL-PARALELO - NRC: 3vo ITIN

DOCENTE : Javier José Cevallos Farías

FECHA DE ENTREGA : 08/06/2026

SANTO DOMINGO - ECUADOR

Contenido

Importancia de la complejidad algorítmica en el desarrollo de software 3
Introducción 3
Objetivos 4
Objetivo General 4
Objetivos Específicos 4
Desarrollo 5
Conclusiones 15
Recomendaciones 16
Referencias 17
Anexos 18

Importancia de la complejidad algorítmica en el desarrollo de software

Introducción

En el contexto del desarrollo de software moderno, donde los sistemas procesan volúmenes de datos cada vez mayores y deben responder en tiempo real, la capacidad de diseñar algoritmos eficientes se ha convertido en una competencia fundamental del ingeniero en sistemas. La complejidad algorítmica es la disciplina que proporciona el marco teórico y las herramientas analíticas para evaluar objetivamente el desempeño de un algoritmo antes de su implementación, con independencia del hardware, el lenguaje de programación y el compilador empleados (Cormen et al., 2022).

Las matemáticas discretas constituyen el soporte formal de la complejidad algorítmica. La teoría de funciones, las relaciones de recurrencia, la combinatoria y la teoría de grafos —temas centrales de la asignatura Modelos Discretos para la Ingeniería— son las herramientas con las cuales se construyen y resuelven las funciones T(n) que describen el comportamiento temporal de los algoritmos (Rosen, 2019). Las funciones de crecimiento —polinomial, logarítmica, exponencial— y el análisis asintótico son conceptos directamente derivados de la matemática discreta.

El análisis de complejidad estudia cómo crecen los recursos computacionales —tiempo de ejecución y espacio de memoria— en función del tamaño de la entrada. Este estudio se formaliza mediante la notación asintótica: un lenguaje matemático preciso que abstrae el comportamiento de los algoritmos de los detalles dependientes del hardware (Sipser, 2013).

Objetivos

Objetivo General

Analizar la importancia de la complejidad algorítmica en el desarrollo de software, mediante el estudio de la notación asintótica, las clases de complejidad computacional y su relación con los modelos matemáticos discretos, con el fin de desarrollar criterios fundamentados para la selección y diseño de algoritmos eficientes en la ingeniería en tecnologías de la información.

Objetivos Específicos

- Definir el concepto de complejidad algorítmica —temporal y espacial— identificando los parámetros que determinan el comportamiento de un algoritmo en función del tamaño de la entrada y relacionándolos con las funciones de crecimiento de Modelos Discretos.

- Comprender y aplicar la notación asintótica Big-O, Omega y Theta como herramienta matemática formal para describir el comportamiento de los algoritmos en el caso peor, mejor y promedio, verificando su aplicación mediante ejemplos resueltos.

- Clasificar los problemas computacionales según las clases de complejidad P, NP, NP-completo y NP-difícil, explorando las implicaciones prácticas de esta clasificación en el diseño de soluciones de software.

Desarrollo

2.3.1 Concepto y Dimensiones de la Complejidad Algorítmica

Definición

La complejidad algorítmica es la disciplina que cuantifica los recursos computacionales que un algoritmo consume en función del tamaño de su entrada, convencionalmente denominado n. Los recursos de interés primarios son el tiempo de ejecución (complejidad temporal) y la memoria utilizada (complejidad espacial). El objetivo es obtener una función matemática T(n) o S(n) que caracterice dicho consumo de forma independiente del hardware, del lenguaje de programación y del compilador empleados (Cormen et al., 2022, p. 3).

Es imprescindible distinguir complejidad algorítmica de rendimiento empírico: el primero es una propiedad intrínseca del algoritmo —una descripción matemática de su comportamiento asintótico—, mientras que el segundo depende del entorno de ejecución. Dos algoritmos con la misma complejidad pueden diferir en rendimiento práctico para entradas pequeñas, pero la complejidad determina cuál escala mejor conforme n crece sin límite (Skiena, 2020).

Complejidad Temporal

La complejidad temporal mide el número de operaciones elementales —comparaciones, asignaciones, accesos a memoria— que ejecuta el algoritmo en función de n. El análisis considera tres escenarios: (a) caso peor (worst case), la entrada que maximiza el tiempo de ejecución —el escenario más útil para garantías en producción—; (b) caso mejor (best case), la entrada más favorable —de valor limitado en la práctica—; y (c) caso promedio

(average case), el tiempo esperado ponderado sobre todas las entradas posibles —más realista, pero matemáticamente más complejo de calcular (Cormen et al., 2022).

Complejidad Espacial

La complejidad espacial mide la cantidad de memoria adicional —excluyendo la entrada— que el algoritmo necesita: variables auxiliares, pilas de recursión y estructuras de datos temporales. Es un factor crítico en sistemas embebidos, aplicaciones móviles o entornos con restricciones de memoria (Skiena, 2020). Un algoritmo puede ser temporalmente eficiente pero espacialmente costoso —como Merge Sort con O(n) de espacio auxiliar—, lo que obliga al ingeniero a evaluar ambas dimensiones según el contexto.

Relación con las Funciones de Crecimiento en Modelos Discretos

La complejidad algorítmica se formaliza directamente a través de las funciones de crecimiento estudiadas en matemáticas discretas. Las relaciones de recurrencia —por ejemplo, $T(n) = 2T(n/2) + n$ para Merge Sort— se resuelven mediante el Teorema Maestro, una herramienta del álgebra discreta. El análisis de bucles anidados utiliza sumatorias y series, mientras que los algoritmos sobre grafos se analizan con teoría de grafos y combinatoria (Rosen, 2019).

2.3.2 Notación Asintótica: O, Ω y Θ

Fundamentos Matemáticos

La notación asintótica es el lenguaje matemático que describe el comportamiento límite de una función T(n) cuando n tiende a infinito. Permite comparar el crecimiento relativo de funciones de complejidad sin importar constantes multiplicativas dependientes del hardware

ni términos de orden menor. Fue introducida formalmente por Bachmann (1894) y popularizada por Landau; Knuth (1997) la estandarizó para el análisis de algoritmos en su obra The Art of Computer Programming.

Notación O Grande — Cota Superior Asintótica

Definición formal: $f(n) = O(g(n))$ si y solo si existen constantes positivas c y n_0 tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. $O(g(n))$ garantiza que el algoritmo nunca consumirá más de $c \cdot g(n)$ pasos para entradas suficientemente grandes. Es la notación más utilizada porque el caso peor es el que se debe garantizar en sistemas productivos (Sipser, 2013).

Notación Ω — Cota Inferior Asintótica

Definición formal: $f(n) = \Omega(g(n))$ si y solo si existen constantes positivas c y n_0 tales que $f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$. Indica el tiempo mínimo garantizado del algoritmo. Es útil para demostrar límites inferiores inherentes: cualquier algoritmo de ordenamiento basado en comparaciones tiene $\Omega(n \log n)$, lo que significa que es matemáticamente imposible ordenar n elementos con menos de n log n comparaciones (Cormen et al., 2022, p. 189).

Notación Θ — Cota Ajustada

Definición formal: $f(n) = \Theta(g(n))$ si y solo si existen constantes c_1, c_2 y n_0 tales que $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ para todo $n \geq n_0$. $\Theta(g(n))$ es la descripción exacta del comportamiento asintótico: el algoritmo crece exactamente como g(n), tanto por arriba como por abajo. Merge Sort tiene $\Theta(n \log n)$ en todos los casos (Cormen et al., 2022).

Tabla 1

Jerarquía de clases de complejidad temporal

Notación	Nombre	Algoritmo ejemplo	Observación práctica
O(1)	Constante	Acceso a arreglo por índice	Independiente del tamaño de entrada
O(log n)	Logarítmica	Búsqueda binaria	Duplicar n añade solo 1 paso
O(n)	Lineal	Búsqueda secuencial	Ideal para recorrido completo de datos
O(n log n)	Linealítmica	Merge Sort, Heap Sort	Óptimo teórico para ordenamiento por comparación
$O(n^2)$	Cuadrática	Bubble Sort, Insertion Sort	Aceptable solo para n < 10 000
$O(n^3)$	Cúbica	Multiplicación matricial naive	Impracticable para n > 1 000
$O(2^n)$	Exponencial	Backtracking sin poda	Solo viable para n < 30
O(n!)	Factorial	TSP fuerza bruta	Inviable para n > 12

Ejemplos Resueltos de Análisis de Complejidad

Búsqueda binaria: el algoritmo divide el espacio a la mitad en cada iteración. Tras k iteraciones quedan $n/2^k$ elementos; el proceso termina cuando $n/2^k = 1$, es decir $k = \log_2(n)$. Por lo tanto $T(n) = \Theta(\log n)$. Para n = 1 000 000 solo se requieren ≈ 20 comparaciones (Cormen et al., 2022).

Bubble Sort: dos lazos anidados de n iteraciones producen $T(n) = n(n-1)/2 \approx n^2/2$, que es $\Theta(n^2)$. Para n = 10 000 implica ≈ 50 000 000 operaciones, evidenciando su ineficiencia para datos de volumen medio.

Merge Sort: la recurrencia $T(n) = 2T(n/2) + cn$ se resuelve por el Teorema Maestro (caso 2) dando $T(n) = \Theta(n \log n)$. Este resultado fundamenta además el límite inferior de cualquier algoritmo de ordenamiento por comparaciones (Cormen et al., 2022, p. 148).

2.3.3 Clases de Complejidad: P, NP, NP-Completo y NP-Difícil

Clase P — Problemas Tratables

Un problema pertenece a P si existe un algoritmo determinista que lo resuelve en tiempo $O(n^k)$ para alguna constante k. Son los problemas tractables: independientemente del exponente, un algoritmo polinomial escala razonablemente para entradas de tamaño práctico (Garey & Johnson, 1979). Ejemplos representativos son el ordenamiento O(n log n), el camino mínimo con el algoritmo de Dijkstra O((V+E) log V) y la prueba de primalidad AKS $O(\log^6 n)$.

Clase NP — Verificables en Tiempo Polinomial

Un problema pertenece a NP (Nondeterministic Polynomial) si, dada una solución candidata, puede verificarse su corrección en tiempo polinomial determinista. Toda clase P está incluida en NP, pues si un problema es resoluble en tiempo polinomial, también es verificable en dicho tiempo. La pregunta abierta P =? NP es considerada el problema abierto más importante de las ciencias de la computación (Sipser, 2013).

Clases NP-Completo y NP-Difícil

Un problema Q es NP-completo si pertenece a NP y todo problema en NP se reduce a Q en tiempo polinomial (reducción de Karp). El primer problema NP-completo fue la satisfacibilidad booleana (SAT), demostrado por Cook en 1971. Si existiera un algoritmo polinomial para cualquier NP-completo, implicaría P = NP. Los problemas NP-difíciles son al menos tan difíciles como los NP-completos, pero no necesariamente verificables en tiempo polinomial (Garey & Johnson, 1979).

Tabla 2

Clasificación de problemas según clase de complejidad

Clase	Definición	Ejemplo	Implicación práctica
P	Resoluble en tiempo polinomial determinista	Ordenamiento, camino mínimo	Algoritmos exactos eficientes disponibles
NP	Verificable en tiempo polinomial	Coloración de grafos, Clique	Sin solución exacta eficiente conocida; se usan aproximaciones
NP-Completo	NP + todo NP reducible a él	SAT, Mochila 0/1, 3-COLOR	Resolver uno resuelve todos; críticos en optimización
NP-Difícil	Al menos tan difícil como NP-completo	TSP optimización, Halt Problem	Puede ser más difícil aún; sin solución polinomial conocida

Estrategias Prácticas para Problemas NP-Completos

- Algoritmos de aproximación: garantizan una solución dentro de un factor α del óptimo; por ejemplo, 2-aproximación para TSP métrico.
- Algoritmos pseudo-polinomiales: eficientes cuando los parámetros numéricos son pequeños; por ejemplo, programación dinámica para mochila 0/1 en O(nW).
- Metaheurísticas: sin garantía de optimalidad pero eficientes en la práctica (algoritmos genéticos, recocido simulado, colonias de hormigas).

2.3.4 Complejidad Algorítmica y Modelos Discretos

Correspondencia entre Herramientas Matemáticas Discretas y Análisis de Algoritmos

La complejidad algorítmica es la aplicación sistemática de los modelos matemáticos discretos al análisis del comportamiento computacional. Cada herramienta del análisis de algoritmos tiene un correlato directo en la matemática discreta (Rosen, 2019):

Tabla 3

Correspondencia entre conceptos de Modelos Discretos y Complejidad Algorítmica

Concepto de Modelos Discretos	Aplicación en Complejidad Algorítmica
Funciones y relaciones	T(n) y S(n) son funciones discretas: tamaño de entrada → recursos consumidos
Sumatorias y series	Análisis de bucles anidados: $\Sigma_{i=1}^{n} i = n(n+1)/2 \rightarrow$ complejidad $O(n^2)$
Relaciones de recurrencia	$T(n) = 2T(n/2) + n$ (Merge Sort), resuelta con el Teorema Maestro → $\Theta(n \log n)$
Logaritmos	La búsqueda binaria divide por 2 en cada iteración: $k = \log_2(n)$ iteraciones → $O(\log n)$
Teoría de grafos	BFS/DFS: O(V+E); Dijkstra: O((V+E) log V); cadenas de Markov en análisis aleatorizado
Combinatoria	Análisis promedio de QuickSort requiere contar permutaciones favorables → O(n log n) esperado
Inducción matemática	Demostración de correctitud de algoritmos recursivos e invariantes de bucle
Lógica proposicional	SAT (primer NP-completo) es directamente un problema de lógica proposicional

Funciones de Crecimiento: del Álgebra Discreta al Análisis de Algoritmos

El estudio de funciones de crecimiento en matemáticas discretas tiene traducción directa a la práctica algorítmica. Una función logarítmica $T(n) = \log n$ corresponde a un algoritmo que descarta la mitad del espacio de búsqueda en cada paso. Una función polinomial $T(n) = n^k$ corresponde a k lazos anidados independientes. Una función exponencial $T(n) = 2^n$ aparece en algoritmos que exploran todos los subconjuntos posibles de un conjunto de n elementos —directamente relacionado con la combinatoria del conjunto potencia de Modelos Discretos (Rosen, 2019, cap. 3).

Esta correspondencia no es accidental: los algoritmos operan sobre estructuras matemáticas discretas (arreglos, grafos, árboles, cadenas), y su análisis requiere las herramientas que estudian esas mismas estructuras. Dominar los modelos discretos es, por tanto, una condición necesaria para analizar algoritmos con rigor (Johnsonbaugh, 2005).

2.3.5 Impacto en el Desarrollo de Software

Rol en el Ciclo de Vida del Desarrollo de Software

La complejidad algorítmica permea todas las fases del ciclo de vida del desarrollo de software (SDLC). En el diseño se seleccionan las estructuras de datos y algoritmos base; en la implementación se escriben las operaciones críticas; en las pruebas de rendimiento se validan los umbrales de escalabilidad; y en el mantenimiento las refactorizaciones algorítmicas reducen costos de infraestructura. Las decisiones algorítmicas determinan la viabilidad del sistema a largo plazo (Fowler, 2018).

Escalabilidad y Rendimiento

El impacto de la complejidad se hace crítico en problemas de escalabilidad. Un sistema de recomendaciones que compara cada usuario con todos los demás —$O(n^2)$— funciona aceptablemente con n = 1 000 usuarios (10^6 operaciones), pero con n = 10 000 000 usuarios requiere 10^{14} operaciones, inviable incluso con hardware de última generación. La solución algorítmica —hashing por similitud, Locality-Sensitive Hashing (LSH)— reduce el problema a O(n), permitiendo escalar a cualquier volumen (Skiena, 2020).

Complejidad de Operaciones por Estructura de Datos

Tabla 4
Complejidad de operaciones según estructura de datos

Estructura de datos	Búsqueda	Inserción	Eliminación	Caso de uso típico
Arreglo ordenado	O(log n)	O(n)	O(n)	Datos estáticos con búsquedas frecuentes

Estructura de datos	Búsqueda	Inserción	Eliminación	Caso de uso típico
Lista enlazada	O(n)	O(1)	O(1)	Inserciones/eliminaciones en cualquier posición
Tabla hash	O(1) prom.	O(1) prom.	O(1) prom.	Cachés, diccionarios, índices en memoria
Árbol AVL / Rojo-Negro	O(log n)	O(log n)	O(log n)	Índices de bases de datos, sistemas de archivos
Montículo (Heap)	O(n)	O(log n)	O(log n)	Colas de prioridad, planificación, Dijkstra
Trie	O(m)*	O(m)*	O(m)*	Autocompletado, búsqueda por prefijo, DNS
Grafo (lista de adyacencia)	O(V+E)	O(1)	O(E)	Redes sociales, mapas, dependencias de módulos

Impacto en Bases de Datos

El diseño de índices en bases de datos relacionales es la aplicación más directa de la complejidad algorítmica en el desarrollo empresarial. Un índice B-Tree reduce la complejidad de búsqueda de O(n) —escaneo completo de tabla— a O(log n), representando la diferencia entre una consulta que tarda horas y una que responde en milisegundos sobre tablas con millones de registros. PostgreSQL, MySQL y MongoDB utilizan internamente estructuras de complejidad logarítmica para garantizar tiempos de consulta predecibles (Ramakrishnan & Gehrke, 2003).

Problema N+1 en APIs y Microservicios

En arquitecturas de microservicios, el problema N+1 ilustra el costo real de ignorar la complejidad: un endpoint ORM que realiza 1 consulta para obtener n entidades y luego n consultas adicionales para sus relaciones tiene complejidad O(n) en llamadas de red, degradando exponencialmente el rendimiento bajo carga. La solución —carga ansiosa (eager loading) o batch queries— reduce este patrón a O(1) llamadas de red con un único query optimizado (Fowler, 2018).

2.3.6 Análisis Comparativo de Algoritmos

Comparativa de Algoritmos de Ordenamiento

Tabla 5

Complejidad temporal y espacial de algoritmos de ordenamiento

Algoritmo	Mejor caso	Caso medio	Peor caso	Espacio	Estable
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Sí
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Sí
Shell Sort	O(n log n)	O(n^1.5)	O(n^2)	O(1)	No
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Sí
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Tim Sort	O(n)	O(n log n)	O(n log n)	O(n)	Sí
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)	Sí

Comparativa de Algoritmos de Búsqueda

Tabla 6

Complejidad de algoritmos de búsqueda y contextos de uso óptimo

Algoritmo	Complejidad	Requisito	Contexto de uso óptimo
Búsqueda lineal	O(n)	Ninguno	Listas pequeñas, datos no ordenados, búsqueda única
Búsqueda binaria	O(log n)	Arreglo ordenado	Búsquedas repetidas en datos ordenados estáticos
Tabla hash	O(1) prom.	Función hash	Cachés, lookups de clave exacta, índices en memoria
BST balanceado	O(log n)	Datos dinámicos	Datos que cambian frecuentemente + búsquedas por rango
BFS / DFS	O(V+E)	Estructura de grafo	Redes, mapas, detección de ciclos, dependencias
Trie	O(m)*	Claves tipo cadena	Autocompletado, búsqueda de prefijos, corrección ortográfica

Impacto Económico de la Elección Algorítmica

Un caso documentado: en 2008, Geisberger et al. publicaron el algoritmo de contracción de jerarquías (CH) para el cálculo de rutas, que redujo el tiempo de consulta en grafos de carretera de varios segundos —con Dijkstra estándar O((V+E) log V)— a microsegundos, manteniendo respuestas exactas. Esta optimización permitió a sistemas como Google Maps escalar de decenas a cientos de millones de consultas diarias sobre la misma infraestructura, representando ahorros estimados de millones de dólares en servidores (Geisberger et al., 2008). El cambio fue puramente algorítmico: misma infraestructura, diferente función de complejidad.

Conclusiones

- La complejidad algorítmica es la herramienta matemática que proporciona el único marco objetivo para comparar la eficiencia de algoritmos con independencia del hardware. Sin este marco, las decisiones de diseño de software carecen de fundamento cuantitativo, conduciendo a sistemas que funcionan en pruebas pero colapsan en producción ante cargas reales. Su dominio es una competencia no negociable del ingeniero en tecnologías de la información.

- La notación asintótica —O, Ω, Θ— no es un formalismo abstracto sino una herramienta práctica de ingeniería que permite predecir el comportamiento de un sistema a escala antes de implementarlo, ahorrando costos de rediseño que pueden superar en órdenes de magnitud el costo de un análisis previo correctamente realizado.

Recomendaciones

- Realizar el análisis de complejidad algorítmica en la fase de diseño —antes de la implementación— como parte obligatoria de la revisión arquitectónica. Identificar la complejidad de los algoritmos centrales evita costosas refactorizaciones posteriores cuando el sistema ya está en producción (Fowler, 2018).

- Incorporar la complejidad como criterio explícito en los code reviews. Toda función que opere sobre colecciones de datos debería documentar su complejidad esperada en el peor caso, especialmente en sistemas de alta concurrencia o que deban escalar horizontalmente (Skiena, 2020).

- Para problemas identificados como NP-completos, explorar sistemáticamente tres alternativas antes de intentar fuerza bruta: algoritmos de aproximación con garantía de ratio, algoritmos pseudo-polinomiales si los parámetros numéricos son acotados, o metaheurísticas cuando se acepta solución de buena calidad sin garantía de optimalidad.

- Complementar el análisis teórico de complejidad con herramientas de profiling empírico para identificar constantes ocultas y patrones de acceso a memoria que el análisis asintótico no captura, logrando una visión integral del rendimiento real del sistema en producción.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.a ed.). MIT Press.

Fowler, M. (2018). Refactoring: Improving the design of existing code (2.a ed.). Addison-Wesley Professional.

Garey, M. R., & Johnson, D. S. (1979). Computers and intractability: A guide to the theory of NP-completeness. W. H. Freeman and Company.

Geisberger, R., Sanders, P., Schultes, D., & Delling, D. (2008). Contraction hierarchies: Faster and simpler hierarchical routing in road networks. En C. C. McGeoch (Ed.), Experimental algorithms (pp. 319–333). Springer. https://doi.org/10.1007/978-3-540-68552-4_24

Johnsonbaugh, R. (2005). Matemáticas discretas (6.a ed.). Pearson Prentice Hall.

Kleinberg, J., & Tardos, É. (2006). Algorithm design. Addison-Wesley.

Knuth, D. E. (1997). The art of computer programming: Vol. 3. Sorting and searching (2.a ed.). Addison-Wesley.

Ramakrishnan, R., & Gehrke, J. (2003). Database management systems (3.a ed.). McGraw-Hill.

Rosen, K. H. (2019). Discrete mathematics and its applications (8.a ed.). McGraw-Hill Education.

Sipser, M. (2013). Introduction to the theory of computation (3.a ed.). Cengage Learning.

Skiena, S. S. (2020). The algorithm design manual (3.a ed.). Springer.

Anexos

Anexo A

Distribución de tareas — Grupo 4

Integrante	Tarea asignada	Estado
Integrante 1	Investigación bibliográfica · Introducción · Objetivos · Secciones 2.3.1 y 2.3.2	✓ Completado
Integrante 2	Secciones 2.3.3, 2.3.4, 2.3.5 y 2.3.6 · Conclusiones · Recomendaciones · Referencias	✓ Completado

Anexo B

Evidencia de elaboración

Se adjuntan capturas de pantalla del proceso de investigación, redacción colaborativa y revisión del documento por parte de los integrantes del Grupo 4, correspondiente al período académico abril–junio 2026.

